

Lecture 6 — Two-Dimensional Viewing and Clipping II: Polygon Clipping, Text Clipping, and the 2D Graphics Pipeline

Lecture focus: Extending clipping from points and lines to polygons, text, GUI regions, and the full 2D graphics pipeline. This lecture focuses especially on the Sutherland–Hodgman polygon clipping algorithm.

Learning outcomes

1. Explain why polygon clipping is needed.
 2. Distinguish line clipping from polygon clipping.
 3. Explain the Sutherland–Hodgman polygon clipping algorithm.
 4. Apply the four inside/outside cases used in polygon clipping.
 5. Explain why clipping can add new vertices to a polygon.
 6. Describe limitations of simple polygon clipping methods.
 7. Explain basic text clipping strategies.
 8. Explain scissor clipping in GUI and graphics APIs.
 9. Describe the full 2D graphics pipeline.
 10. Relate clipping to CAD, maps, browsers, games, vector editors, and printers.
-

1. Where Lecture 6 fits

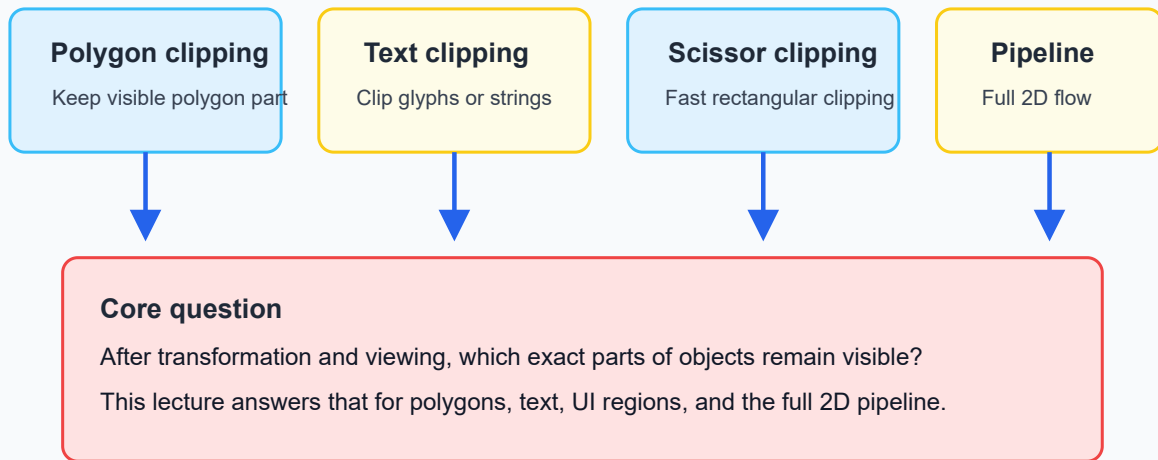
Lecture 5 introduced the basic viewing idea:

```
world window → clipping → viewport mapping → screen
```

We discussed point clipping and line clipping. But real scenes usually contain more than isolated points and lines. They contain polygons, text, icons, filled areas, GUI panels, map regions, and vector shapes.

That is why we now extend the discussion.

2D Viewing and Clipping II



The main question in this lecture is:

After the viewing window is chosen, how do we keep only the visible part of complex 2D objects?

2. Why polygon clipping is needed

A polygon may be partly inside and partly outside the clipping window.

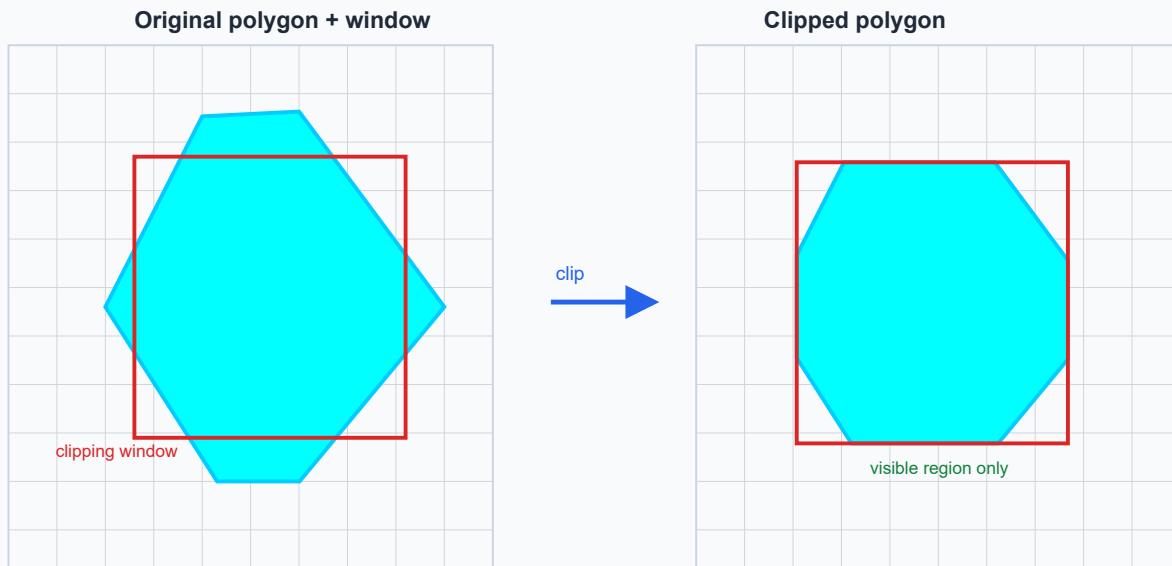
If we simply reject the whole polygon because one vertex is outside, we may remove visible content.

If we draw the whole polygon without clipping, it may draw outside the visible region.

So the correct solution is:

Construct a new polygon representing only the visible part.

Polygon clipping: before and after



Example applications

Polygon clipping appears in:

- map rendering,
- CAD drawings,
- vector graphics editors,
- game UI panels,
- browser rendering,
- printing systems,
- 2D and 3D graphics pipelines.

Important difference from line clipping

A line segment can be clipped into at most one visible segment inside a rectangular window.

A polygon, however, may:

- lose vertices,
- gain new intersection vertices,
- change its shape,
- sometimes break into multiple pieces depending on the polygon and clipping region.

For the first polygon clipping algorithm, we focus on a common and important case: clipping a polygon against a **convex rectangular window**.

3. The Sutherland–Hodgman polygon clipping algorithm

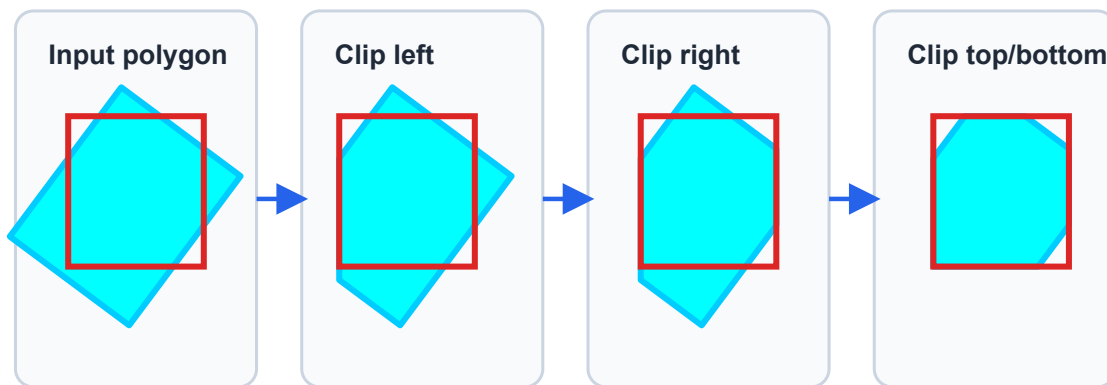
The **Sutherland–Hodgman algorithm** is a classic polygon clipping algorithm.

It clips a polygon against one clipping boundary at a time.

For a rectangular clipping window, the boundaries are:

1. left,
2. right,
3. bottom,
4. top.

Sutherland–Hodgman polygon clipping: edge-by-edge idea



The output from one boundary becomes the input to the next boundary.

The key idea is:

```
input polygon
  ↓ clip against left boundary
intermediate polygon
  ↓ clip against right boundary
intermediate polygon
  ↓ clip against bottom boundary
intermediate polygon
```

```
↓ clip against top boundary  
final clipped polygon
```

The output of one boundary becomes the input to the next boundary.

Why this is elegant

Instead of solving the whole polygon-window problem at once, we solve four simpler problems:

```
Clip the polygon against one boundary line.
```

This divide-and-process style makes the algorithm easier to implement and explain.

4. Inside and outside for one clipping edge

For one clipping boundary, every point is either inside or outside.

For example, suppose we are clipping against the left boundary:

```
x = xmin
```

A point is inside if:

```
x ≥ xmin
```

For the right boundary:

```
x ≤ xmax
```

For the bottom boundary:

```
y ≥ ymin
```

For the top boundary:

```
y ≤ ymax
```

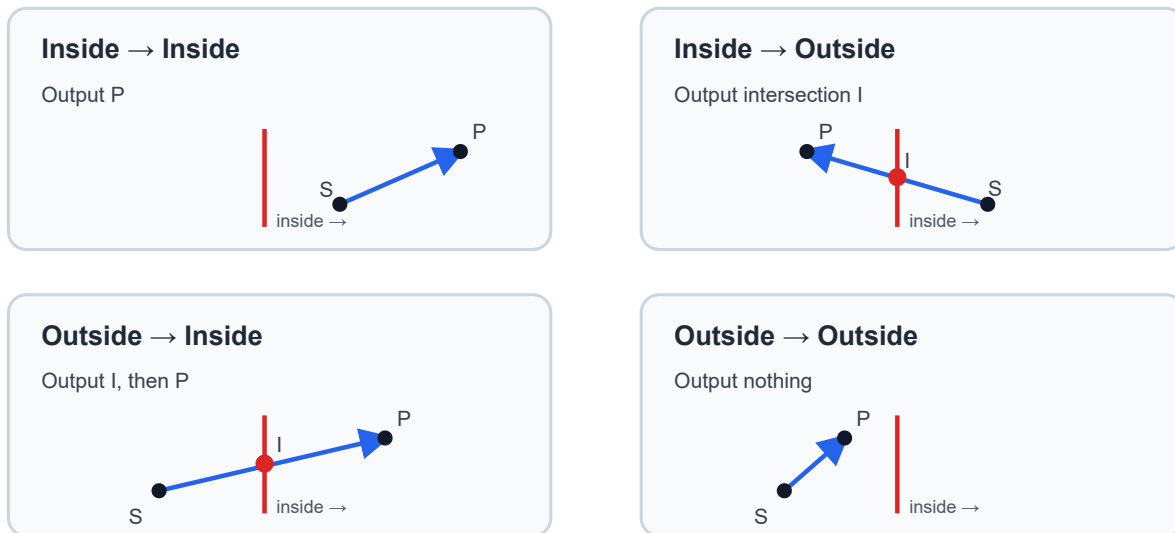
These inequalities assume the usual mathematical coordinate convention. Screen coordinates may reverse top and bottom tests depending on the y-axis direction.

5. The four cases in polygon clipping

When clipping a polygon against one boundary, we examine each polygon edge from previous vertex **S** to current vertex **P**.

There are four cases.

Polygon clipping cases for one edge $S \rightarrow P$



Case 1: Inside $\rightarrow$ Inside

Both **S** and **P** are inside.

Output:

P

Reason: the current endpoint remains visible.

Case 2: Inside $\rightarrow$ Outside

The edge leaves the visible region.

Output:

intersection point **I**

Reason: the edge is visible only until it crosses the boundary.

Case 3: Outside → Inside

The edge enters the visible region.

Output:

```
intersection point I  
P
```

Reason: the visible part starts at the boundary intersection and continues to P.

Case 4: Outside → Outside

Both endpoints are outside.

Output:

```
nothing
```

Reason: for a convex clipping boundary, this edge segment contributes no visible part for that boundary.

6. Sutherland–Hodgman pseudocode

A simplified version is:

```
SutherlandHodgman polygonClip(polygon, clipWindow):  
    outputList = polygon  
  
    for each clipping boundary edge E:  
        inputList = outputList  
        outputList = empty  
  
        S = last vertex in inputList  
  
        for each vertex P in inputList:  
            if P is inside E:  
                if S is inside E:  
                    output P  
                else:
```

```
        output intersection(S, P, E)
        output P
    else:
        if S is inside E:
            output intersection(S, P, E)
        else:
            output nothing

    S = P

return outputList
```

What is the algorithm doing?

It walks around the polygon edge by edge. For each edge, it decides what part of that edge survives the current clipping boundary.

Why `S = last vertex` initially?

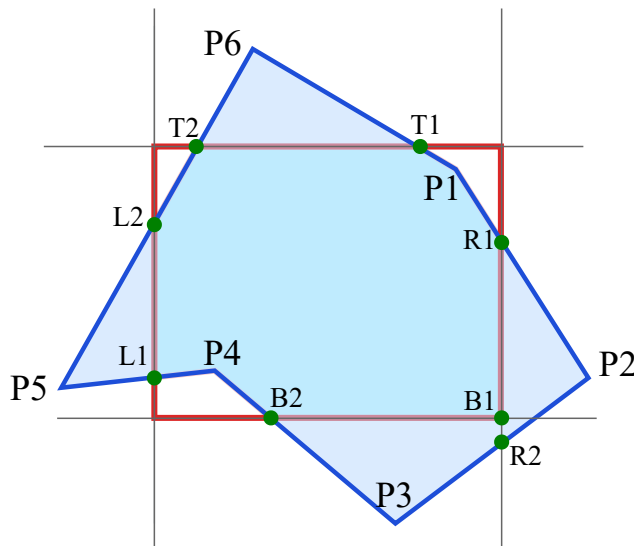
A polygon is closed. The edge from the last vertex to the first vertex is also part of the polygon.

So before processing the first vertex, we treat the previous vertex as the last vertex of the polygon.

7. Worked idea: clipping against a rectangular window

The diagram below shows a polygon being clipped against a rectangle.

Worked idea: clipping a polygon against a rectangular window



The final polygon may have more vertices than the original because intersections are added.

1. List polygon vertices (P1 - P6)
2. Clip against left edge
 $P6 > P1 > P2 > P3 > P4 > L1 > L2 > P6$
3. Clip output against right edge
 $P6 > P1 > R1 > R2 > P3 > P4 > L1 > L2 > P6$
4. Clip output against bottom edge
 $P6 > P1 > R1 > B1 > B2 > P4 > L1 > L2 > P6$
5. Clip output against top edge
 $T1 > P1 > R1 > B1 > B2 > P4 > L1 > L2 > T2 > T1$
6. Final polygon is visible part

During clipping:

1. original vertices may disappear,
2. new intersection vertices may be added,
3. the final polygon boundary follows part of the original polygon and part of the clipping window.

Important observation

The clipped polygon can have **more vertices** than the original polygon.

For example, a triangle clipped by a rectangle may become a quadrilateral or pentagon.

8. Small numerical example

Suppose we clip a polygon edge from:

$S = (2, 3)$
 $P = (8, 6)$

against the left boundary:

$x = 5$

Here:

```
S is outside because  $2 < 5$   
P is inside because  $8 \geq 5$ 
```

So this is:

```
Outside → Inside
```

We must output:

```
intersection I, then P
```

To find the intersection, use the line parameter:

$$X(t) = S + t(P - S)$$

For x-coordinate:

$$x(t) = 2 + t(8 - 2) = 2 + 6t$$

Set $x(t) = 5$:

$$\begin{aligned}2 + 6t &= 5 \\6t &= 3 \\t &= 0.5\end{aligned}$$

Now y-coordinate:

$$y(t) = 3 + 0.5(6 - 3) = 4.5$$

So:

$$I = (5, 4.5)$$

The output for this edge is:

$$(5, 4.5), (8, 6)$$

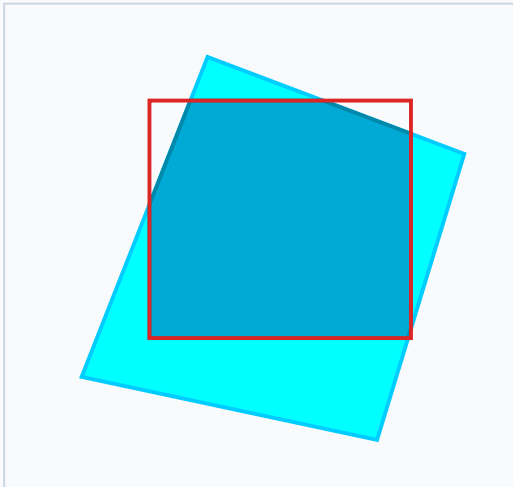
9. Limitations and special cases

The basic Sutherland–Hodgman algorithm works naturally when the clipping window is convex.

A rectangle is convex, so it works well for normal rectangular clipping windows.

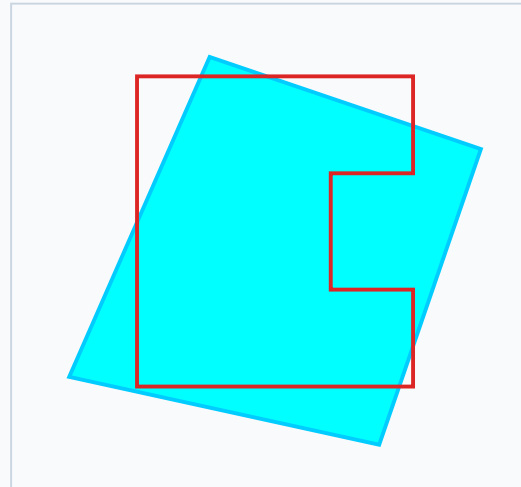
Convex vs concave clipping windows

Convex clipping window



Sutherland–Hodgman works naturally for convex windows.

Concave clipping window



Concave windows require more careful algorithms.

One simple edge-by-edge inside test is not enough.

What if the clipping region is concave?

Concave clipping regions are more difficult. A simple inside/outside test against each edge may not produce the expected result.

For concave clipping windows or complex polygons, other algorithms may be needed.

One historically important algorithm is **Weiler–Atherton clipping**, which can handle more complex polygon cases.

What if the polygon is concave?

Sutherland–Hodgman can clip concave subject polygons against a convex clipping window in many practical cases. However, care is needed when the output may need to represent multiple disconnected pieces.

For an undergraduate first treatment, remember this:

Sutherland–Hodgman is simple and excellent for clipping polygons against convex windows,

especially rectangles.

10. Why clipping should happen before filling

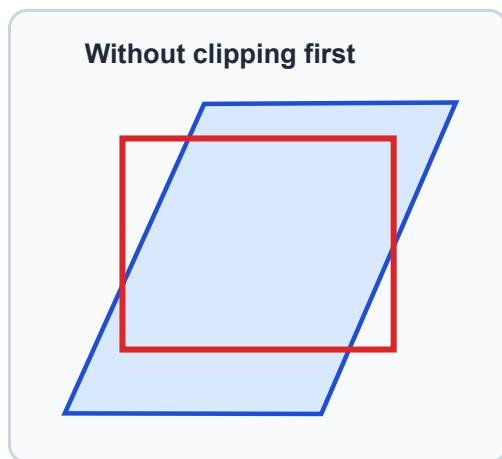
In raster graphics, filled polygons are eventually converted to pixels.

If we fill before clipping, the fill operation may affect pixels outside the visible region.

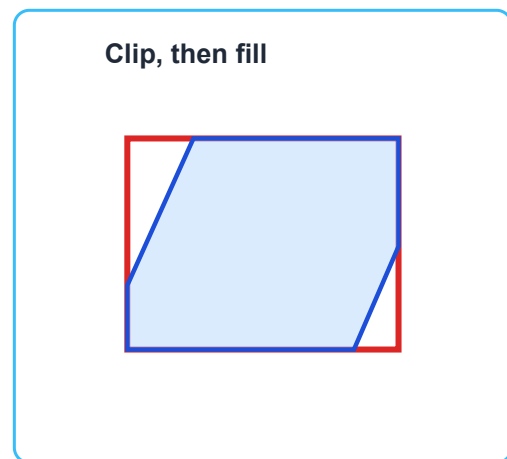
So a common order is:

```
transform polygon → clip polygon → fill/rasterize polygon
```

Why clip before filling?



Fill may cover pixels outside the intended visible window.



The filled result is limited to the visible region.

This order keeps the final image correct and avoids unnecessary work.

11. Text clipping

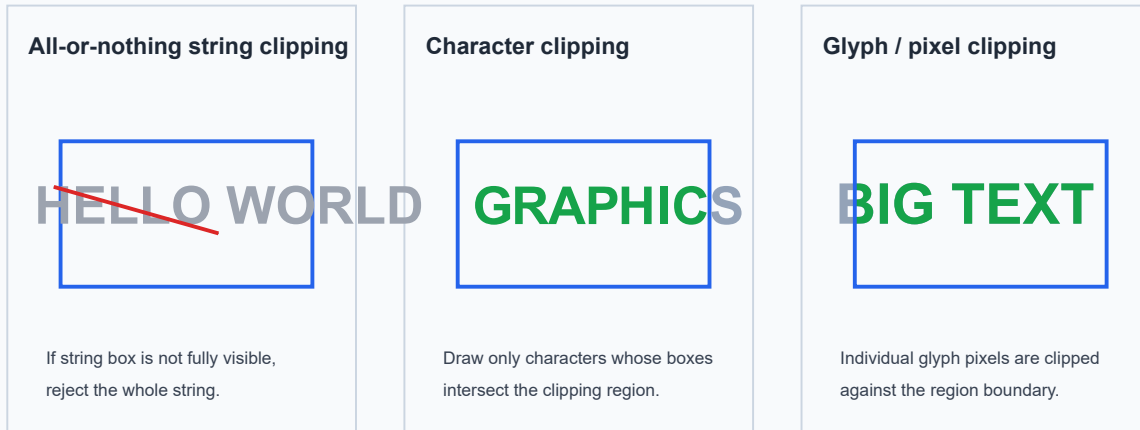
Text is also a graphical object. It must often be clipped inside a window, panel, page, or viewport.

For example:

- a text editor clips text inside the editor area,
- a browser clips text inside a card,
- a button clips long labels,

- a PDF viewer clips text to page boundaries.

Text clipping strategies



There are several strategies.

Strategy 1: All-or-nothing string clipping

The whole string is accepted or rejected based on its bounding box.

This is simple but crude.

If part of the string is outside, the entire string may disappear.

Strategy 2: Character clipping

Each character is tested separately.

Characters outside the clipping region are rejected, while visible characters are drawn.

This is better than all-or-nothing clipping but still may be rough at boundaries.

Strategy 3: Glyph or pixel clipping

The actual pixels or vector outline of each glyph are clipped.

This gives the best visual result, especially when only part of a character is visible.

Modern text rendering systems often use sophisticated versions of this idea.

Trivia

Text clipping is one reason high-quality UI rendering is harder than it looks. A simple label inside a small button may involve font layout, glyph shaping, anti-aliasing, subpixel positioning, and clipping.

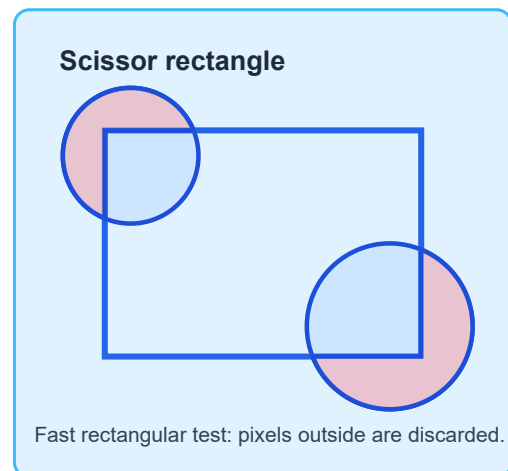
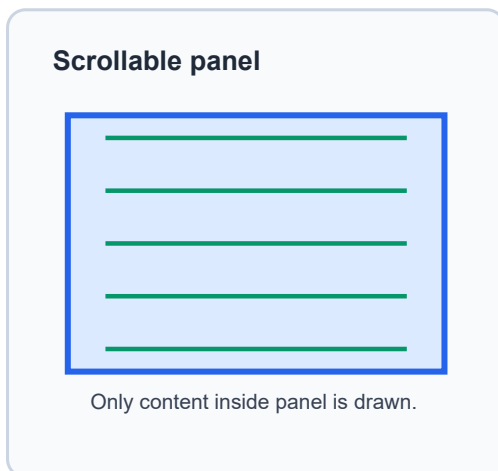
12. Scissor clipping

A very common practical clipping method is rectangular clipping, often called a **scissor test** or **scissor clipping**.

The idea is simple:

Only draw pixels inside a given rectangular region.

Scissor clipping in GUI and graphics APIs



This is heavily used in:

- GUI rendering,
- game UI,
- browser layout,
- OpenGL and other graphics APIs,
- scrollable panels,
- split-screen rendering,
- minimaps.

Example

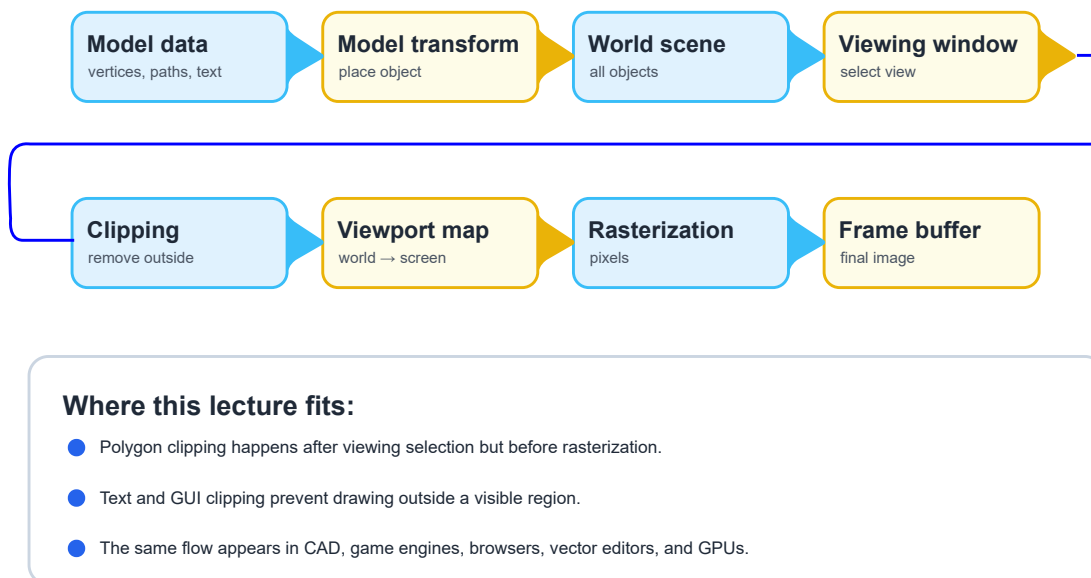
A scrollable list may have many items, but only the items inside the panel should be visible.

The scissor rectangle prevents drawing outside the panel.

13. The complete 2D graphics pipeline

We can now combine the ideas from Lectures 3, 4, 5, and 6.

Complete 2D graphics pipeline



A typical 2D graphics pipeline is:

```
Model data
↓
Model transformation
↓
World scene
↓
Viewing/window selection
↓
Clipping
↓
Viewport mapping
↓
Rasterization / scan conversion
```

↓
Frame buffer
↓
Display

Where each lecture fits

Lecture 3:

Rasterization / scan conversion

Lecture 4:

Transformations

Lecture 5:

Window, viewport, point clipping, line clipping

Lecture 6:

Polygon clipping, text clipping, full 2D pipeline

14. Coordinate spaces in 2D graphics

A graphics system often uses multiple coordinate spaces.

Model or local coordinates

The object is defined in its own local space.

Example:

a button icon drawn around (0, 0)

World coordinates

Objects are placed into a larger scene.

Example:

a map symbol placed at a geographic location

Viewing coordinates

A selected region of the world is chosen by the window.

Device or screen coordinates

The final location is expressed in pixels or device units.

Why multiple spaces are useful

They keep the system organized. We can design an object once, place it many times, choose a view, clip invisible parts, and finally draw it to the screen.

15. Applications of polygon and region clipping

Applications of polygon and region clipping

Vector editors

Shapes clipped inside masks and artboards.

CAD

Large drawings are clipped to the active view.

Games

Camera and UI panels clip world and HUD geometry.

Browsers

CSS overflow and clip-path use clipping concepts.

GIS

Administrative regions and map tiles are clipped to view.

Printers

Page boundaries clip text, images, and vector paths.

Trivia: a clipping path in design software is a direct artistic use of a graphics algorithm idea.

Vector graphics editors

Shapes may be clipped inside masks, artboards, or other shapes.

CAD

Large engineering drawings are clipped to the current view.

Games

World geometry, minimaps, and UI panels all use clipping ideas.

Browsers

CSS properties such as `overflow: hidden` and `clip-path` are practical versions of clipping concepts.

GIS and maps

Map regions and map tiles are clipped to the current viewport.

Printers and PDF systems

Text, images, and vector paths must be clipped to page boundaries.

16. Common mistakes

1. Thinking polygon clipping is the same as line clipping.
 2. Forgetting that clipping can create new vertices.
 3. Forgetting the closing edge from the last vertex to the first vertex.
 4. Mixing up `S` and `P` in Sutherland–Hodgman.
 5. Outputting the wrong vertices in the four inside/outside cases.
 6. Assuming Sutherland–Hodgman works for every possible clipping situation without care.
 7. Filling a polygon before clipping when the fill should be restricted.
 8. Thinking text clipping is always done one character at a time.
 9. Confusing viewport mapping with clipping.
 10. Forgetting that screen y-axis direction may differ from mathematical y-axis direction.
-

17. Short exam-style questions

1. Why is polygon clipping needed?
2. What is the main idea of the Sutherland–Hodgman algorithm?
3. List the four inside/outside cases for clipping an edge from `S` to `P`.
4. What is output for the case Outside → Inside?
5. Why can a clipped polygon have more vertices than the original polygon?
6. What type of clipping window is Sutherland–Hodgman especially suitable for?
7. What is text clipping?
8. What is scissor clipping?

9. Why is clipping usually performed before rasterization?
10. Draw the major stages of a 2D graphics pipeline.
-

18. Practice problems

Problem 1

A polygon edge goes from:

```
S = (1, 3)
P = (7, 5)
```

The clipping boundary is:

```
x = 4
```

The inside region is:

```
x ≥ 4
```

Classify the edge and find the output of this edge after clipping.

Problem 2

For each case, write what should be output in Sutherland–Hodgman clipping:

```
Inside → Inside
Inside → Outside
Outside → Inside
Outside → Outside
```

Problem 3

A triangle is clipped against a rectangle. Is it possible for the final clipped polygon to have four or five vertices? Explain.

Problem 4

Explain why a scrollable panel in a GUI needs clipping.

Problem 5

Write the main stages of the 2D graphics pipeline from model data to display.

19. Lecture summary

Lecture 6 summary

- | | |
|-----------------------------|---|
| ● Polygon clipping | Find visible part of a polygon inside a clipping region. |
| ● Sutherland–Hodgman | Clip polygon edge-by-edge against a convex window. |
| ● Four cases | Inside/inside, inside/outside, outside/inside, outside/outside. |
| ● Text clipping | Reject strings, clip characters, or clip glyph pixels. |
| ● Scissor clipping | Fast rectangular clipping used heavily in GUIs and APIs. |
| ● Pipeline | Model → transform → view → clip → viewport → rasterize. |
| ● Key warning | Clipping can add new vertices at intersections. |

Big idea: clipping decides the final visible geometry before rasterization.

The big takeaways are:

- Polygon clipping constructs a visible polygon from a possibly partly visible polygon.
- Sutherland–Hodgman clips a polygon against one boundary at a time.
- Each edge from **S** to **P** falls into one of four inside/outside cases.
- Clipping may add intersection vertices.
- Sutherland–Hodgman is especially suitable for convex clipping windows such as rectangles.
- Text clipping can be done at the string, character, glyph, or pixel level.
- Scissor clipping is a fast rectangular clipping method used in GUIs and graphics APIs.
- The full 2D pipeline connects modeling, transformation, viewing, clipping, viewport mapping, and rasterization.

20. Final trivia box

- The word **clip** appears in design tools, browser engines, GPU APIs, and video editing software.
- A clipping path in vector design software is a direct artistic use of polygon clipping ideas.

- Scissor clipping is one of the simplest but most useful optimizations in real-time graphics.
- When a browser hides overflowing content inside a card or panel, it is using the same core idea taught in this lecture.
- Polygon clipping is also a stepping stone toward 3D clipping, hidden-surface processing, and modern rendering pipelines.